এখন তোমার CT Kidney Dataset (Normal, Cyst, Tumor, Stone) দিয়ে Google Colab-এ একটি 4-class image classification model train করব।

আমি প্রথমে EfficientNetB0 Transfer Learning ব্যবহার করার পরামর্শ দিচ্ছি। এটি pretrained CNN, তাই শুরু থেকে model train করার তুলনায় কম data ও computational resources দিয়ে ভালো baseline তৈরি করা সম্ভব।

সম্পূর্ণ training workflow

01

Colab GPU setup ও library import

02

Google Drive থেকে split dataset load

03

Train, validation ও test dataset verify

04

Image preprocessing ও augmentation

05

EfficientNetB0 model তৈরি

06

Model training (transfer learning)

07

Fine-tuning ও training graphs

08

Test evaluation, confusion matrix, classification report

09

Model ও results Google Drive-এ save

তোমার dataset-এর verified তথ্য:

Dataset	Images
Training	8,350
Validation	1,789
Testing	1,790
Total	11,929

একটি গুরুত্বপূর্ণ গবেষণাগত সীমাবদ্ধতা: exact duplicates বাদ দেওয়া হয়েছে, কিন্তু patient identifiers না থাকায় একই রোগীর ভিন্ন CT slices আলাদা split-এ আছে কি না নিশ্চিত করা যায়নি। তাই ভবিষ্যতে model accuracy রিপোর্ট করার সময় এটিকে limitation হিসেবে উল্লেখ করবে।

Colab-এ যাও: Runtime → Change runtime type → Hardware accelerator → T4 GPU (যদি available থাকে) → Save।

এরপর নিচের code-গুলো আলাদা আলাদা cell-এ run করবে। আগের মতো প্রতিটি step-এর output আমাকে দিলে আমি check করে পরবর্তী ধাপে guide করতে পারব।

Step 1 — GPU ও TensorFlow check

নতুন cell-এ run করো:

import tensorflow as tfimport numpy as npimport pandas as pdimport matplotlib.pyplot as pltimport osimport randomSEED = 42tf.keras.utils.set_random_seed(SEED)print("TensorFlow version:", tf.__version__)print("Available GPUs:", tf.config.list_physical_devices("GPU"))

Expected: GPU list-এ অন্তত একটি GPU দেখা যাবে। `[]` এলে GPU activate হয়নি।

Step 2 — Google Drive mount

from google.colab import drivedrive.mount("/content/drive")print("Google Drive connected successfully!")

আগে mount করা থাকলে `Drive already mounted` আসতে পারে। সেটাও ঠিক আছে।

Step 3 — ZIP থেকে dataset extract

import zipfilefrom pathlib import PathZIP_PATH = Path("/content/drive/MyDrive/kidney_dataset_split_70_15_15.zip")DATASET_DIR = Path("/content/kidney_training_dataset")assert ZIP_PATH.exists(), "ZIP file not found in Google Drive!"if not DATASET_DIR.exists(): DATASET_DIR.mkdir(parents=True) with zipfile.ZipFile(ZIP_PATH, "r") as zip_ref: zip_ref.extractall(DATASET_DIR)print("Dataset ready!")print("Location:", DATASET_DIR)for split in ["train", "validation", "test"]: print(split, (DATASET_DIR / split).exists())

তিনটি split-এর জন্য `True` আসতে হবে।

Step 4 — Image dataset load

এখানে image size `224 × 224` এবং batch size `32` ব্যবহার করব।

import tensorflow as tfIMG_SIZE = (224, 224)BATCH_SIZE = 32SEED = 42CLASS_NAMES = ["Cyst", "Normal", "Stone", "Tumor"]train_ds = tf.keras.utils.image_dataset_from_directory(DATASET_DIR / "train", labels="inferred", label_mode="categorical", class_names=CLASS_NAMES, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=True, seed=SEED)val_ds = tf.keras.utils.image_dataset_from_directory(DATASET_DIR / "validation", labels="inferred", label_mode="categorical", class_names=CLASS_NAMES, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False)test_ds = tf.keras.utils.image_dataset_from_directory(DATASET_DIR / "test", labels="inferred", label_mode="categorical", class_names=CLASS_NAMES, image_size=IMG_SIZE, batch_size=BATCH_SIZE, shuffle=False)print("Classes:", train_ds.class_names)

Expected:

Found 8350 files belonging to 4 classes.
Found 1789 files belonging to 4 classes.
Found 1790 files belonging to 4 classes.

Classes: ['Cyst', 'Normal', 'Stone', 'Tumor']

Step 5 — Sample images visualize

Training শুরুর আগে images ও labels ঠিক আছে কি না দেখব।

plt.figure(figsize=(12, 8))for images, labels in train_ds.take(1): for i in range(12): ax = plt.subplot(3, 4, i + 1) plt.imshow(images[i].numpy().astype("uint8")) label_index = np.argmax(labels[i].numpy()) plt.title(CLASS_NAMES[label_index]) plt.axis("off")plt.tight_layout()plt.show()

এখানে চারটি class-এর kidney CT images দেখতে পাবে। সব class একটি batch-এ থাকা বাধ্যতামূলক নয়।

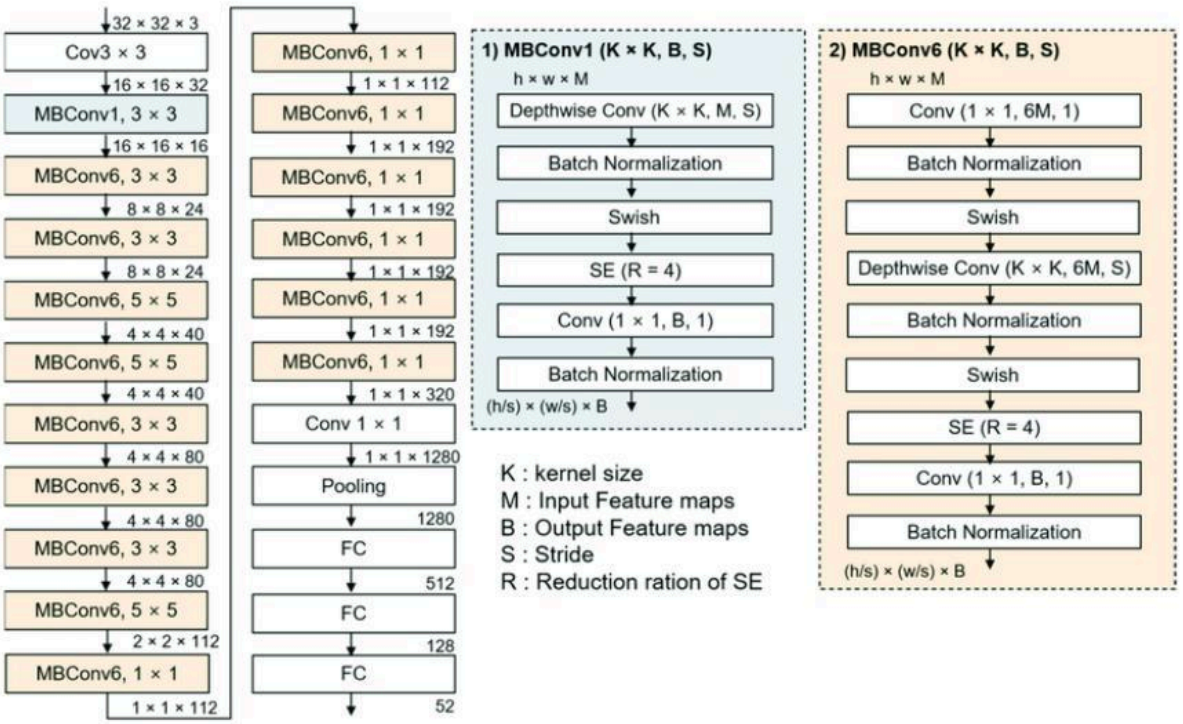
Step 6 — Data augmentation ও performance optimization

Augmentation শুধু training-এর সময় প্রয়োগ হবে। Validation ও test images অপরিবর্তিত থাকবে।

from tensorflow.keras import layersdata_augmentation = tf.keras.Sequential([layers.RandomRotation(0.05), layers.RandomZoom(0.10), layers.RandomTranslation(0.05, 0.05)], name="data_augmentation")AUTOTUNE = tf.data.AUTOTUNEtrain_ds = train_ds.prefetch(AUTOTUNE)val_ds = val_ds.prefetch(AUTOTUNE)test_ds = test_ds.prefetch(AUTOTUNE)print("Dataset pipeline ready!")

CT images-এর জন্য আপাতত horizontal/vertical flip ব্যবহার করছি না, কারণ anatomical orientation পরিবর্তন গবেষণার ক্ষেত্রে গুরুত্বপূর্ণ হতে পারে।

Step 7 — EfficientNetB0 model তৈরি



EfficientNetB0

ImageNet-pretrained convolutional backbone ব্যবহার করব। এর ওপর global average pooling, dropout এবং চারটি output neuron যোগ করব।

from tensorflow.keras import layers, modelsbase_model = tf.keras.applications.EfficientNetB0(include_top=False, weights="imagenet", input_shape=(224, 224, 3))base_model.trainable = Falseinputs = layers.Input(shape=(224, 224, 3))x = data_augmentation(inputs)# EfficientNetB0 already includes input rescaling.x =

base_model(x, training=False)x = layers.GlobalAveragePooling2D()(x)x = layers.Dropout(0.3)(x)outputs = layers.Dense(4, activation="softmax")(x)model = models.Model(inputs, outputs)model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001), loss="categorical_crossentropy", metrics=["accuracy"])model.summary()

গুরুত্বপূর্ণ: EfficientNetB0-এর Keras implementation-এ input rescaling অন্তর্ভুক্ত থাকে। তাই আলাদাভাবে pixel values `/255` করবে না।

Step 8 — Model training (Phase 1)

প্রথমে pretrained backbone freeze রেখে classification head train করব।

from tensorflow.keras.callbacks import (EarlyStopping, ModelCheckpoint, ReduceLROnPlateau)MODEL_DIR = Path("/content/drive/MyDrive/Kidney_Model_Results")MODEL_DIR.mkdir(parents=True, exist_ok=True)callbacks = [EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True), ModelCheckpoint(filepath=str(MODEL_DIR / "best_phase1.keras"), monitor="val_loss", save_best_only=True), ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2, min_lr=1e-6)]history = model.fit(train_ds, validation_data=val_ds,

Training-এর সময় এরকম output আসবে:

Epoch 1/20
... accuracy: ... loss: ... val_accuracy: ... val_loss: ...
Epoch 2/20
...

Accuracy কত হবে আগে থেকে বলা যায় না। Validation loss এবং train–validation performance-এর পার্থক্য দেখে model-এর আচরণ বুঝতে হবে।

Step 9 — Fine-tuning (Phase 2)

Phase 1 শেষ হলে pretrained model-এর শেষের কিছু layers unfreeze করব।

base_model.trainable = True# Freeze all but the last 30 layersfor layer in base_model.layers[:-30]: layer.trainable = False# Keep BatchNormalization layers frozenfor layer in base_model.layers: if isinstance(layer, tf.keras.layers.BatchNormalization): layer.trainable = Falsemodel.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5), loss="categorical_crossentropy", metrics=["accuracy"])fine_callbacks = [EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True), ModelCheckpoint(filepath=str(MODEL_DIR / "best_finetuned.keras"), monitor="val_loss", save_best_only=True), ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=2,

Fine-tuning শেষ হলে validation results দিয়ে Phase 1 ও Phase 2-এর মধ্যে ভালো checkpoint নির্বাচন করবে। Test set দিয়ে model নির্বাচন করবে না।

Step 10 — Training accuracy ও loss graph

train_acc = (history.history["accuracy"] + fine_history.history["accuracy"])val_acc = (history.history["val_accuracy"] + fine_history.history["val_accuracy"])train_loss = (history.history["loss"] + fine_history.history["loss"])val_loss = (history.history["val_loss"] + fine_history.history["val_loss"])plt.figure(figsize=(9, 5))plt.plot(train_acc, label="Training Accuracy")plt.plot(val_acc, label="Validation Accuracy")plt.axvline(len(history.history["accuracy"]) - 1, linestyle="--", color="gray", label="Fine-tuning starts")plt.xlabel("Epoch")plt.ylabel("Accuracy")plt.title("EfficientNetB0 Training and Validation Accuracy")plt.legend()plt.grid(alpha=0.3)plt.savefig(MODEL_DIR / "accuracy_curve.png", dpi=300)plt.show()plt.figure(figsize=(9, 5))plt.plot(train_loss, label="Training Loss")plt.plot(val_loss, label="Validation Loss")plt.xlabel("Epoch")plt.ylabel("Loss")plt.title("EfficientNetB0 Training and Validation Loss")plt.legend()plt.grid(alpha=0.3)plt.savefig(MODEL_DIR / "loss_curve.png", dpi=300)plt.show()

Step 11 — Best checkpoint নির্বাচন ও test evaluation

Validation loss দিয়ে checkpoint নির্বাচন করব। এরপর প্রথমবারের মতো held-out test set evaluate করব।

from tensorflow.keras.models import load_modelphase1_model = load_model(MODEL_DIR / "best_phase1.keras")phase2_model = load_model(MODEL_DIR / "best_finetuned.keras")phase1_val_loss, phase1_val_acc = phase1_model.evaluate(val_ds, verbose=0)phase2_val_loss, phase2_val_acc = phase2_model.evaluate(val_ds, verbose=0)if phase2_val_loss < phase1_val_loss: best_model = phase2_model selected_phase = "Fine-tuned"else: best_model = phase1_model selected_phase = "Phase 1"print("Selected model:", selected_phase)test_loss, test_accuracy = best_model.evaluate(test_ds)print("Test Loss:", round(test_loss, 4))print("Test Accuracy:", round(test_accuracy * 100, 2), "%")

Step 12 — Classification report

Accuracy ছাড়াও precision, recall এবং F1-score গুরুত্বপূর্ণ, বিশেষ করে class imbalance থাকলে।

from sklearn.metrics import classification_reporty_true = []y_pred = []for images, labels in test_ds: predictions = best_model.predict(images, verbose=0) y_true.extend(np.argmax(labels.numpy(), axis=1)) y_pred.extend(np.argmax(predictions, axis=1))report = classification_report(y_true, y_pred, target_names=CLASS_NAMES, digits=4, zero_division=0)print(report)with open(MODEL_DIR / "classification_report.txt", "w") as f: f.write(report)

Step 13 — Confusion matrix

from sklearn.metrics import ConfusionMatrixDisplayimport matplotlib.pyplot as pltfig, ax = plt.subplots(figsize=(8, 7))ConfusionMatrixDisplay.from_predictions(y_true, y_pred, display_labels=CLASS_NAMES, cmap="Blues", values_format="d", ax=ax)plt.title("EfficientNetB0 Confusion Matrix")plt.tight_layout()plt.savefig(MODEL_DIR / "confusion_matrix.png", dpi=300, bbox_inches="tight")plt.show()

এখানে কোন class-এর image কোন class হিসেবে ভুল predict হয়েছে, সেটা বুঝতে পারবে।

Step 14 — Trained model save

best_model.save(MODEL_DIR / "Kidney_EfficientNetB0_Final.keras")print("Model saved successfully!")print("Location:", MODEL_DIR)

গবেষণার জন্য যেসব **results** রাখবে

- Training ও validation accuracy/loss curves
- Test accuracy এবং test loss
- Per-class precision, recall ও F1-score
- Macro এবং weighted average F1-score
- Confusion matrix
- Training configuration: image size, batch size, epochs, optimizer, learning rate ও random seed
- Dataset splitting, duplicate removal এবং patient-level leakage limitation

আরও একটি গুরুত্বপূর্ণ বিষয়: test data-তে augmentation করবে না এবং test accuracy দেখে বারবার model hyperparameter পরিবর্তন করবে না। এতে final evaluation biased হতে পারে।

এখন তোমার করণীয়

সব code একসঙ্গে run করার দরকার নেই। আমরা আগের মতো output দেখে এগোব।

প্রথমে Step 1 run করো এবং GPU output আমাকে পাঠাও। আমি বলব হ্যাঁ—next step অথবা না—আগে সমস্যা ঠিক করতে হবে।

এরপর Step 2, Step 3 এভাবে এগোব। এতে training শুরু হওয়ার আগেই configuration বা dataset-loading error থাকলে ধরতে পারব।